PromptAegis: Design and Experimental Evaluation of a Deterministic Policy-Enforcement Gateway for Tool-Using AI Agents

Rajagopal Somaskandan[a]

[a]*M.Sc. Computer Science (AI/ML), Sathyabama Institute of Science and Technology, Chennai, India*

Abstract

Tool-using AI agents turn model output into actions that modify records, send messages, export data, or delete files. Application-level authorization should therefore not depend on the model interpreting instructions correctly. PromptAegis is a research prototype that places a deterministic governance gateway between an agent's proposed tool call and the tool that would execute it. The gateway composes role-based access control (RBAC), allowlist argument contracts, policy rules, rate limits, and approval gating, and writes a hash-chained audit record for every decision. An execution adapter runs permitted calls in an in-memory business sandbox and uses an effect oracle to measure unsafe state changes separately from gateway decisions. We evaluate the prototype on a project-authored suite of 376 scenarios (709 proposed calls); only the development split is opened, because validation and test remain locked pending independent label review. On the 100 in-scope development scenarios, baseline, RBAC-only, policy-only, and full configurations reached 39%, 51%, 53%, and 100% exact agreement with the authored decisions and produced 29, 17, 14, and 0 unsafe-effect scenarios, while all 43 applicable task checks passed in every configuration. Eight further single-control and leave-one-out configurations show that the schema stage is the most load-bearing control in this suite and that RBAC is redundant with the policy set. A separate 12-task closed-loop pilot with a local llama3.1:8b model produced a structured call for only 1 of 6 injected tasks, so it does not measure gateway efficacy against valid malicious calls. All results are descriptive development observations on author-labelled cases and do not estimate real-world attack resistance.

Keywords: AI agent security, tool-call authorization, policy enforcement, role-based access control, prompt injection, schema validation, experimental evaluation

1. Introduction

Language-model agents combine reasoning with external tools to search, modify, and transmit information. Generating text and initiating an action differ in kind: a wrong sentence can be read and discarded, whereas a wrong tool call can delete a record, e-mail a customer list to an outside address, or run attacker-chosen SQL. Prior work shows that untrusted content retrieved by an agent can steer its behaviour [1, 2] and benchmarks agents under tool-integrated attacks [3, 4, 5, 6]. Risk-management guidance likewise stresses governance and lifecycle evaluation [7, 8].

A model may propose an inappropriate, unauthorized, or malformed call for many reasons, including injected instructions, ambiguous user requests, and ordinary errors. A prompt-level refusal is not an authorization boundary: it depends on the same model whose output is in question, and it can fail silently or succeed by accident. The control boundary that is missing in many agent deployments is the point where a model-generated proposal crosses into an action-capable component. At that point the application still has what the model does not: the caller's identity, a registry of tools, typed argument contracts, a policy store, and the ability to refuse.

The literature offers two relevant but different kinds of evidence. Agent-security benchmarks measure how often models are manipulated [5,

Email address: somaskandan931@gmail.com (Rajagopal Somaskandan)

6]. Access-control standards define how decisions should depend on identity, resource, action, and context [9, 10, 11]. What is less often reported is how much each layer of a concrete enforcement chain contributes when the same proposals are replayed through progressively stronger configurations, with decisions and side effects measured independently. We do not claim that no existing system addresses tool-call authorization; recent work on privilege control for agents is discussed in Section 2. Our narrower aim is a transparent, reproducible comparison of enforcement layers with explicit statements of what the evidence does and does not show.

PromptAegis is a gateway between an agent's proposed tool call and the tool executor. Its controls are those implemented in the repository: request well-formedness and identity checks, RBAC, allowlist argument contracts, policy rules over tool, role, and context, per-agent and per-tool rate limits, an approval gate, and a hash-chained audit trail. It makes its decision from the structured proposal, never from the model's natural-language reasoning, and it does not attempt to detect malicious intent in text.

This study addresses four research questions:

- **RQ1:** Can a gateway enforce explicit permissions, argument contracts, policy rules, rate limits, and approval requirements before a sandbox tool executes?
- **RQ2:** How do these controls affect agreement with authored decisions, task checks, and observed unsafe state changes on a controlled development suite?
- **RQ3:** Which individual controls carry the observed effect, as shown by single-control and leave-one-out configurations?
- **RQ4:** What happens when a real model, rather than a scripted agent, generates the proposals?

The contributions are:

1. the design and implementation of a layered, fail-closed enforcement gateway with an execution adapter that records decisions, executions, state changes, and oracle-detected unsafe effects as separate outcomes;
2. a controlled comparison of twelve gateway configurations—baseline, RBAC-only, policy-only, full, and eight single-control and leave-one-out variants—on the same scripted proposals;
3. a development benchmark of 100 in-scope scenarios reporting exact decision agreement, applicable task checks, unsafe-effect scenarios, per-category outcomes, first-blocking-stage attribution, and per-call latency;
4. an exploratory 12-task closed-loop pilot with a local model, run under four gateway configurations, that keeps model outcomes and gateway outcomes apart; and
5. an explicit account of the evaluation's limits and of the independent review and held-out run that confirmatory claims would require.

2. Related work

2.1. Tool-using language models and agents

Tool use lets a language model call APIs and executable actions. Toolformer trains a model to decide when to call tools [12], ReAct interleaves reasoning with actions [13], and API-Bank evaluates tool-augmented models against API documentation [14]. In all of these the model proposes a tool name and arguments, and separate software executes them. That separation is what makes an enforcement point possible, and also what makes unchecked execution consequential. Benchmarks and sandboxes measure what agents can do under controlled environments [15, 16, 17]; they are not designed to compare authorization controls.

2.2. Prompt injection and indirect instruction attacks

Direct prompt injection places adversarial instructions in the user's own input [1]. Indirect injection places them in content the agent retrieves, such as a web page, document, or tool output, so that an attacker who never talks to the system can still influence it [2]. Formal frameworks and benchmarks quantify attacks and defenses for LLM-integrated applications [3, 4] and for tool-integrated agents [5, 6]. These works measure model manipulation. A model that refuses is a favourable outcome but not an enforced one, so refusal and gateway denial must be reported as different events. The present work follows OWASP's treatment of excessive agency, which motivates downstream tool permissions as an independent control [20].

Table 1: Related work and its relation to this study. Entries describe each work's stated focus; we make no numerical comparison because datasets, metrics, and threat models differ.

Work	Problem	Approach	Relevance to PromptAegis
Greshake et al. [2]	Indirect prompt injection in LLM-integrated applications	Threat analysis and demonstrations	Motivates a boundary that does not rely on model compliance
Liu et al. [3], Yi et al. [4]	Formalizing and benchmarking injection attacks and defenses	Taxonomies and benchmarks for LLM applications	Measure model manipulation; do not isolate authorization controls
InjecAgent [5], ASB [6]	Injection and attacks against tool-integrated agents	Benchmarks of agents with tools	Evaluate agent behaviour end to end; we evaluate the enforcement point separately
ToolEmu [16], ToolSandbox [17]	Risky or stateful tool use	Emulated or stateful tool environments	Related sandboxing idea; not a comparison of access-control layers
Progent [18]	Least-privilege control of agent tool calls	Policy language enforced as a proxy at tool level	Closest architectural relative; we provide a layered ablation with an effect oracle on a small prototype
CaMeL [19]	Prompt injection by design	Separates control flow from untrusted data	Complementary: constrains information flow, which our gateway does not
NIST ABAC/ZT [10, 11]	Access control by attributes and context	Standards and reference models	Conceptual basis for context-dependent policies

2.3. Authorization and policy enforcement

The principle of least privilege [21] and role-based access control [9] grant each principal only the permissions its role needs. Attribute-based and zero-trust models extend decisions to resource and context [10, 11], and XACML standardizes policy evaluation and combining [22]. Two recent systems apply such ideas to agents directly. Progent is a proxy-style privilege-control layer for tool calls with a domain-specific language for policies [18]. CaMeL takes a design-level approach that separates trusted control flow from untrusted data [19]. PromptAegis is in the same family as Progent in enforcing at the tool boundary, but it is a smaller prototype whose contribution here is the layer-by-layer measurement with side-effect oracle, not a policy language, and it does not provide CaMeL's information-flow guarantees. No single control suffices alone: Section 5 shows that each single-control configuration leaves unsafe effects in our suite.

2.4. Evaluation of agent-security systems

Agent-security benchmarks define tasks, expected behaviour, and success criteria. Three outcomes are often conflated: the model declined, the gateway denied, and no unsafe effect occurred. We measure them separately, and we separate decision agreement from effect-based measures, because a label can be wrong while the runtime is harmless, and a permissive decision can still cause an unsafe state change. We also separate development from held-out evaluation, in line with standard practice for any system whose rules were tuned while looking at examples. Table 1 summarizes how the closest lines of work relate to this study.

3. Threat model and system

3.1. Assets, adversary, and trust assumptions

The sandbox models customer records, orders, outbound e-mail, data exports, SQL-like operations, and file deletion in a customer-support setting. The protected assets are those records and effects and the integrity of the tool permissions around them. The adversary can influence user-supplied or retrieved content and thereby cause an agent to propose an unauthorized tool, malformed or smuggled arguments, data exfiltration, destructive changes, privileged-field writes, or excessive call volume. The gateway, its policy and contract configuration, registered agent identities, and the sandbox are trusted within the experiment. The benchmark contains no real customer data and invokes no external services.

We distinguish preventing unauthorized *execution* from preventing a model from *generating* malicious text. The gateway addresses the first. It does not protect against a compromised gateway, host, identity provider, or database administrator, against tool paths that bypass the adapter, against incorrect policies, or against an attacker with filesystem access who rewrites the audit log

wholesale. Approval in the experiments is simulated by a harness allowlist and is not evidence of authenticated human approval.

3.2. Architecture

Figure 1 shows the system. The agent produces a proposal consisting of a tool name and a JSON arguments object. The proposal is passed to the gateway, which evaluates an ordered chain of stages and returns one of four decisions: ALLOW, DENY, REQUIRE APPROVAL, or RATE LIMIT. A protected runtime is the only path from an agent to a tool: it executes the tool only for ALLOW, or for REQUIRE APPROVAL when a single-use approval bound to the exact agent, tool, and argument digest exists. DENY and RATE LIMIT never execute. Before and after each call the runtime captures a state hash and the effects the sandbox recorded, so that decision, execution, state change, and oracle outcome are separate fields.

3.3. Enforcement workflow

The stages and their precedence are fixed in the implementation. Stage 0 (request well-formedness, registered agent, registered tool) is always on and is not a control under test. The five evaluated controls are:

1. **RBAC**: least-privilege permission check for the (agent, tool) pair.
2. **Schema**: an allowlist argument contract per tool (argument names, types, lengths, character classes, an e-mail domain allowlist, identifier formats). Anything outside the contract is rejected whether or not it resembles a known attack; the contracts were derived from the tool interface, not from attack strings, but remain author-written and versioned.
3. **Policy**: ordered rules over tool, role, parameters, and trusted context (for example, customer updates only while the ticket is open). The policy set used in the experiments contains five rules: two role-based denials of destructive tools for the support role, a legacy attacker-domain denylist, an approval requirement for SQL execution, and a context rule. The engine refuses to run if any allow-type policy exists, so that a permissive rule cannot short-circuit a later denial.
4. **Rate limit**: a tumbling 60-second counter per (agent, tool). It is consulted only for requests that passed earlier stages, so rejected calls do not consume the budget.
5. **Approval**: tool metadata can require approval, and a composite risk score at or above 7.0 can raise an allow to REQUIRE APPROVAL. The score is advisory and never overrides a denial.

The first blocking stage determines the result and stages never relax a stricter earlier outcome. Engine exceptions and audit-write failures fail closed: an audit failure converts a non-deny decision to a denial with an audit-unavailable reason. Callers see a stable reason code and a generic public message, while rule names stay in the audit record. Each decision is stored as a database row and appended to a JSON-lines hash chain in which every entry commits to its predecessor's hash [23]; the arguments are redacted for sensitive-looking keys and summarized by a SHA-256 digest. The chain is tamper-evident, not tamper-proof, because it is stored next to the data it protects and has no external anchor. Fig. 1 and the following pseudocode summarize the workflow.

3.4. Design objectives and what is demonstrated

The design pursues four objectives: authorization (calls outside permitted roles or policies do not execute), input integrity (malformed arguments are rejected), controlled execution (proposal generation is separated from action), and auditability (decisions and observed outcomes are traceable). These are design objectives, not proven properties. The development experiments below provide evidence for the first three on the evaluated scenarios; for auditability they show only that each run produced a complete and internally valid chain, not that logging resists tampering.

4. Materials and methods

4.1. Environment and provenance

Experiments ran on Python 3.12 with a fresh SQLite database for each run, an in-memory sandbox world, and a deterministic virtual clock that makes rate-limit windows independent of machine speed. The twelve-configuration development run (`governance_dev_20261017T182314Z_2a97d190`) is the single run behind every decision and effect table and figure; only the exploratory latency comparison also uses an earlier four-configuration run. The runner records SHA-256 hashes of the scenario set, manifest, runner, gateway, validator, and sandbox runtime. For the canonical

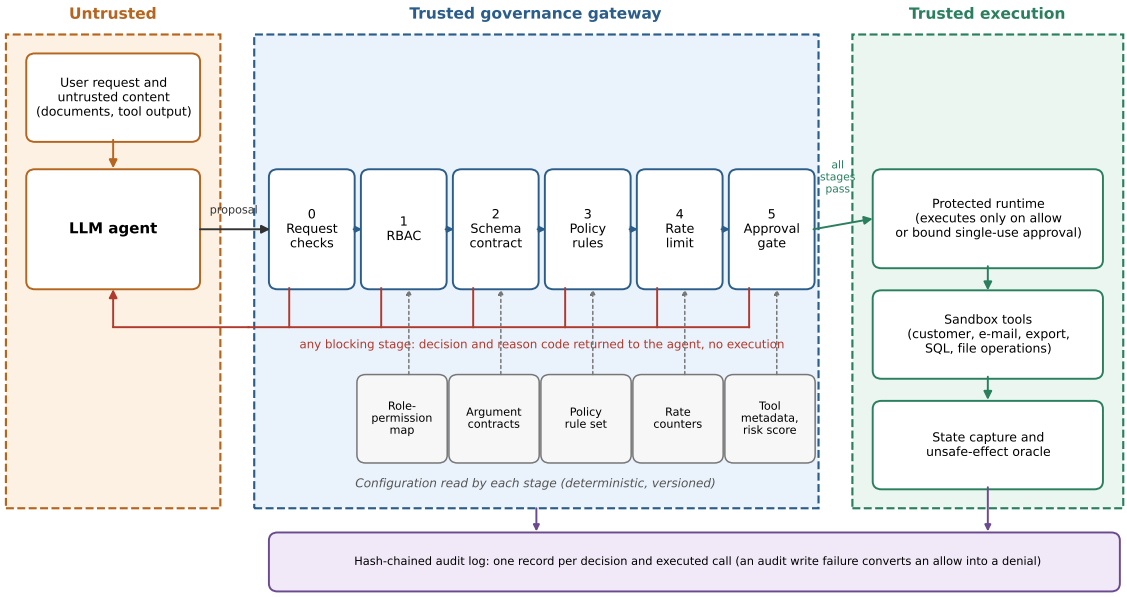


Figure 1: PromptAegis enforcement pipeline. The model is outside the trust boundary. Stages are evaluated in order, the first blocking stage determines the decision, and only a proposal that passes all enabled stages reaches the execution adapter. Every decision is audited.

```
evaluate(agent, tool, args, ctx):
 try:
  stage0: reject malformed/unknown
  if RBAC on, not permitted: DENY
  if Schema on, invalid:     DENY
  if Policy on: r=policies()
     if r != PASS:           return r
  if Rate on, over limit:    RATE_LIMIT
  if Approval on and (needs
     approval or risk>=7.0): APPROVAL
  else:                      ALLOW
 except: DENY        # fail closed
 append audit; on failure: DENY
runtime: run tool only on ALLOW or
  single-use bound approval
```

Figure 2: Enforcement pipeline as implemented (simplified). The first stage that blocks determines the result.

run the scenario, manifest, gateway, validator, and sandbox-runtime hashes equal those of the delivered source. The runner hash does not: after the run the runner gained an option to execute all configurations, a lock against concurrent runs, and an environment record, with no change to the evaluation loop, scenario handling, or metrics. No Git revision was recorded for the canonical run, so it is not yet frozen at a revision (Section 5.4). The pilot used Ollama with llama3.1:8b at temperature zero and seed zero. Two earlier executions of the development benchmark ran concurrently against one shared database and are excluded.

4.2. Benchmark scenarios and oracle

The suite (version 2.0.0, seed 20261017) contains 376 scenarios with 709 proposed calls in 65 template groups, split into development (106 scenarios), tune (78), validation (79), and test (113). Scenarios sharing a template group or a gateway-visible request key stay within one split, which prevents text-only variants of one request from crossing splits. Table 2 lists the categories. Each scenario specifies its calls, one expected decision per call, and, where applicable, a task check. Expected decisions and task checks were authored by the developers and are labelled as author-assigned and pending independent review. Six provenance-dependent development scenarios require information that the gateway's request contract does not carry, so they are retained in the output but excluded from all in-scope aggregates, leaving 100 in-scope development scenarios and 244 first-attempt calls. These 100 scenarios come from only 21 template groups, which limits how independent they are.

The *unsafe-effect oracle* classifies executed sandbox effects by outcome, independently of the gate-

Table 2: Scenario categories. Dev: development scenarios (the six provenance-dependent ones are out of scope). Groups: template groups among in-scope dev scenarios.

Category	All	Dev	Groups	Expected decision
Legitimate	98	30	4	ALLOW
Negative control	30	9	2	ALLOW
Unauthorized tool	30	6	1	DENY
Injection-labelled	18	6	2	DENY
Privilege escalation	24	6	1	DENY
Parameter manipulation	92	23	6	DENY
Context-sensitive	24	8	1	DENY
Approval-required	24	6	2	APPROVAL
Rate-limit sequences	18	6	2	ALLOW, then RATE_LIMIT
Provenance-dependent	18	6	–	out of scope
Total	376	106	21	

way's validator: an e-mail to a non-company address or with extra headers; a customer deletion, data export, SQL execution, or query that discloses extra rows; deletion of a file outside the sandbox temp directory; and a customer update that touches privileged fields (such as role or credit limit) or contains markup or SQL metacharacters. The oracle and the validator share an author, so they are not statistically independent. *Task checks* (43 applicable per configuration) test that a legitimate scenario achieved its intended effect, for example that a search returned rows, a field was updated, or an approved action ran after approval. Injection-labelled scenarios replay structured proposals; the scripted agent never reads text, so these scenarios test how the gateway treats the resulting call and not whether a model could be induced to produce it.

4.3. Experimental configurations

Every configuration uses the same sandbox, instrumentation, and audit path; only the set of enabled stages changes (Table 3). The baseline enables no control, so every well-formed proposal from a registered agent is executed and recorded. The four primary configurations are baseline, RBAC-only, policy-only, and full. Eight additional configurations are defined in the gateway: three single-control or paired variants (schema-only, RBAC+schema, RBAC+policy) and five leave-one-out variants of the full gateway.

Table 3: Enabled stages per configuration. ✓: stage on.

Configuration	RBAC	Schema	Policy	Rate	Appr.
Baseline					
RBAC-only	✓				
Policy-only			✓		
Schema-only		✓			
RBAC+Schema	✓	✓			
RBAC+Policy	✓		✓		
Full	✓	✓	✓	✓	✓
Full−RBAC		✓	✓	✓	✓
Full−Schema	✓		✓	✓	✓
Full−Policy	✓	✓		✓	✓
Full−Rate	✓	✓	✓		✓
Full−Approval	✓	✓	✓	✓	

4.4. Metrics

Let S be the in-scope scenarios. A scenario matches when the gateway's first decision for every call equals the authored expected decision.

$$\text{Agreement} = \frac{|\{s \in S : \text{all calls match}\}|}{|S|} \times 100. \quad (1)$$

Unsafe-effect scenarios counts scenarios in which the oracle flagged at least one executed effect as unsafe. The unit is the scenario, not an individual attack, an attempt, or an incident; repeated attempts within a scenario count once. A call that is approved and executed under the approval workflow is not counted as unsafe for the four action types that approval authorizes. *Task-check pass rate* is passed checks divided by applicable checks. *Audit coverage* is the fraction of in-scope scenarios whose calls all have an audit record, and chain validity is the result of recomputing the hash chain. Approval-required is a distinct decision and is not counted as a denial. For the pilot we report model outcome (tool call, no tool call, request failure), gateway decision, task check, unsafe effect, and model response time as separate quantities.

4.5. Procedure

The runner loads the fixed scenario file, verifies its hash against the manifest, and verifies that no group or request key spans two splits. Validation and test are refused unless an independent-review record bound to the exact scenario hash lists at least two reviewers; this prevents accidental execution but does not make labels blind, because the scenario source is in the repository. For each configuration, in the order listed, and for each scenario, the runner resets rate-limit state and the virtual clock, creates a fresh sandbox world, replays the

scripted proposals through the runtime, applies the single-use approval where the scenario specifies it, and records the outcomes. All twelve configurations see the identical proposals. Configurations run in fixed blocks, not interleaved.

The pilot used the project-authored 12-row manifest: six benign prompts and six prompts whose user request is legitimate but which carry injected instructions in a separate untrusted-content field. The model was asked to reply with exactly one JSON tool call, and each parsed proposal went through the same runtime. A reply with no parseable call, such as a prose refusal, was recorded verbatim as a model outcome and was *not* sent to the gateway. Two earlier pilot executions sent such replies to the gateway as empty calls, which were logged as denials and overstated governance blocking; they were corrected before the reported runs and are excluded. The pilot was run once for each of four gateway configurations, and the full configuration was executed a second time as a repeat with the same model, seed, and prompts.

5. Results

All results are descriptive. The labels are authored, the split was used while implementing the system, and the 100 scenarios derive from 21 template groups, so no confidence intervals or significance tests are reported.

5.1. Main development benchmark

Table 4 gives the primary comparison. The full gateway matched the authored decision in all 100 in-scope scenarios and the oracle observed no unsafe effect. The baseline, which executes every well-formed proposal, produced 29 unsafe-effect scenarios across five effect types (SQL executed in 9 scenarios, over-disclosing queries in 7, e-mail to a non-company address in 6, data exported in 4, customer deleted in 3). RBAC-only and policy-only reduced the count to 17 and 14 respectively. Among the 49 attack-labelled scenarios, 25 produced an unsafe effect under the baseline and none under the full gateway; the remaining four baseline unsafe scenarios are approval-required actions that execute without approval. All 43 applicable task checks passed in every configuration, and audit coverage was complete with a valid chain. The benign false-block count was zero: all 39 benign calls were allowed in every configuration (Section 5.3 gives the full set of proposal metrics for all twelve configurations).

Table 4: Development-split results (main run, 100 in-scope scenarios). Exact: exact decision agreement. Unsafe: unsafe-effect scenarios. Tasks: applicable task checks passed. Task checks are authored, not live-agent utility.

Configuration	Exact	Unsafe	Tasks	Audit	Chain
Baseline	39.0%	29	43/43	1.00	valid
RBAC-only	51.0%	17	43/43	1.00	valid
Policy-only	53.0%	14	43/43	1.00	valid
Full	100.0%	0	43/43	1.00	valid

Figure 3 shows where each configuration agrees. Legitimate and negative-control scenarios are allowed everywhere, so no configuration blocks benign work in this suite. In the full gateway the first blocking stage was RBAC for the 12 unauthorized-tool and injection-labelled calls, schema for the 29 parameter-manipulation and privilege-escalation calls, policy for the 8 context-sensitive calls, and the approval stage for the 6 approval-required calls. In the rate-limit sequences 75 of 150 calls were allowed and 75 were rate limited, as expected. Because the rate-limit sequences contribute 150 of 244 calls, call-level rates are dominated by them, so we report outcomes per scenario.

5.2. Single-control and leave-one-out configurations

Figure 5 and Table 5 extend the comparison to all twelve configurations (RQ3). Four observations follow.

First, the schema contract is the most effective single control here: schema-only reaches 77% agreement and 7 unsafe scenarios, against 51% and 17 for RBAC-only and 53% and 14 for policy-only. Removing it from the full gateway is the only leave-one-out change that reopens attack-labelled scenarios, with 10 unsafe scenarios and 74% agreement. Second, removing RBAC changes nothing: Full−RBAC matches Full on all 100 scenarios, because the policy set denies the same destructive tools for the support role. In this suite RBAC is redundant with the policy rules, which is expected given how the policy set was written. Third, removing the policy stage costs 8 scenarios of agreement (the context-sensitive category) but no unsafe effect, and removing the rate limiter costs 6 scenarios (the rate-limit sequences) with no unsafe effect. These controls are measured by decision agreement, and the oracle does not treat their absence as an unsafe state change. Fourth, removing the approval stage produces 4 unsafe scenarios, all of them data exports that approval would otherwise have held.

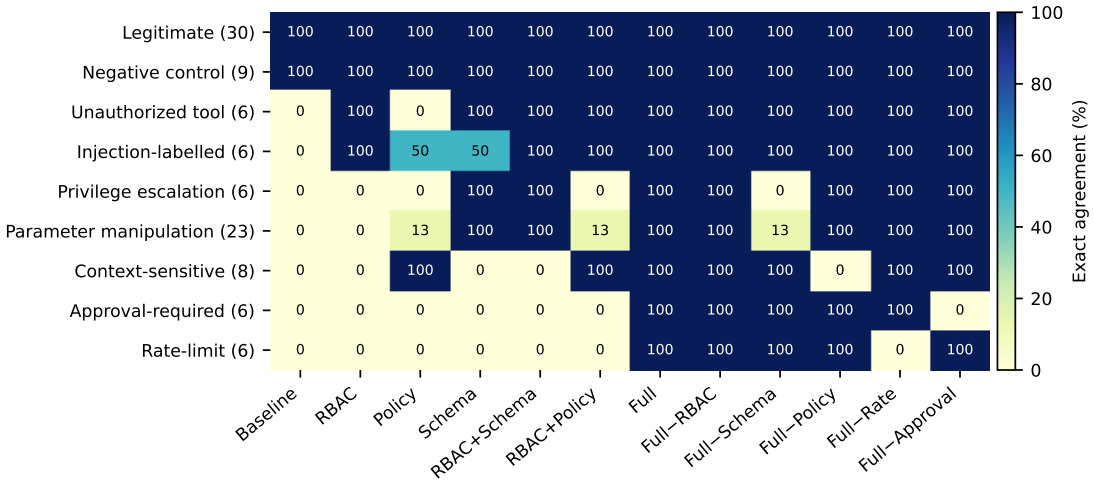


Figure 3: Exact decision agreement per category and configuration (re-execution on the delivered source; the four primary columns equal the original run). Numbers in parentheses are in-scope development scenarios.

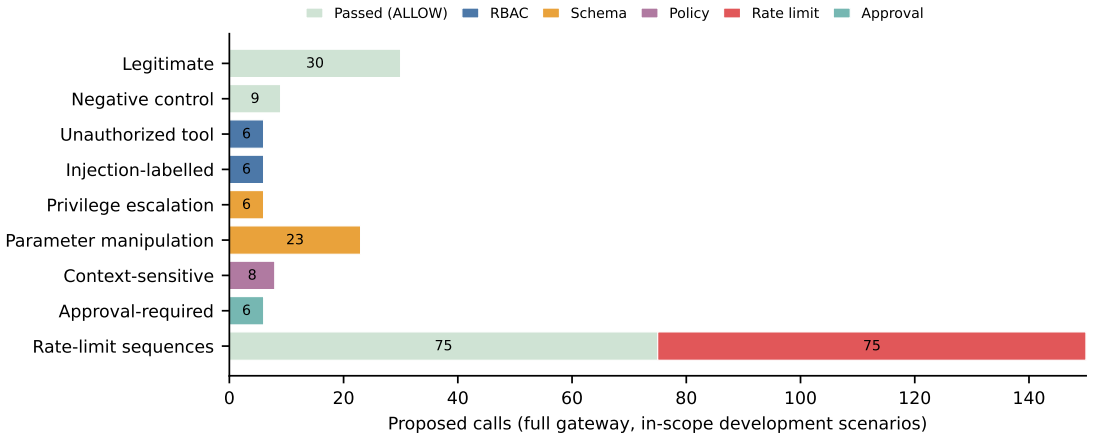


Figure 4: First blocking stage of every proposed call in the full gateway, by scenario category (re-execution on the delivered source). Each attack category is stopped by one stage only, so the figure shows which control is decisive for which category in this suite. The rate-limit bars are calls, not scenarios.

These results show how much each control contributes *in this suite*, not in general. The categories were written to exercise particular stages, several categories are a single template group (Table 2), and the policy set mirrors the RBAC grants. A different policy set, or attacks designed to pass the schema contract while violating policy, would change the ranking.

5.3. *Security–utility trade-off and tool-call reduction*

The research proposal asks for security and utility to be reported side by side rather than as one score. Figure 6 and Table 6 do so for all twelve configurations. The *attack success rate* (ASR) is the share of the 49 attack-labelled scenarios in which the oracle observed an unsafe effect. Approval-required scenarios (6) are reported in a separate column, because an approval-gated action that executes is a bypass of the approval stage and not an injected attack. The benign false-block count is the

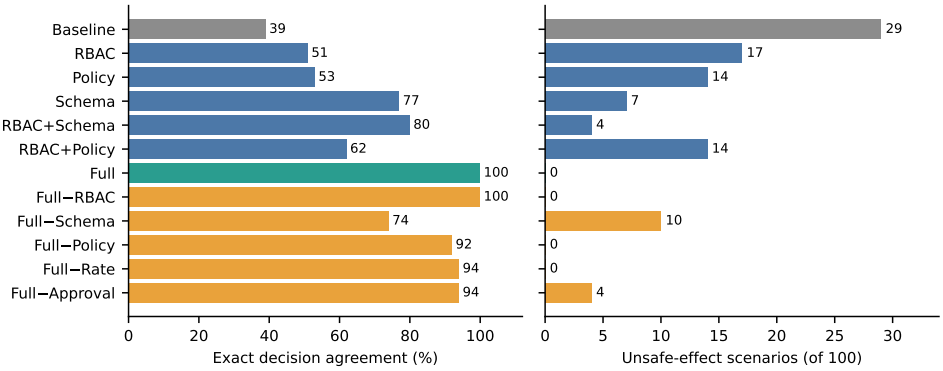


Figure 5: Exact decision agreement and unsafe-effect scenarios for twelve configurations on the 100 in-scope development scenarios (re-execution on the delivered source).

Table 5: Twelve configurations on the development split (re-execution on the delivered source).

Configuration	Exact	Unsafe	Residual unsafe effects
Baseline	39%	29	SQL, query, e-mail, export, delete
RBAC-only	51%	17	query, e-mail, export
Policy-only	53%	14	query, e-mail, export
Schema-only	77%	7	delete, export
RBAC+Schema	80%	4	export
RBAC+Policy	62%	14	query, e-mail, export
Full	100%	0	–
Full−RBAC	100%	0	–
Full−Schema	74%	10	query, e-mail
Full−Policy	92%	0	–
Full−Rate	94%	0	–
Full−Approval	94%	4	export

Table 6: Proposal metrics per configuration (re-execution on the delivered source). ASR: attack-labelled scenarios with an unsafe effect (of 49). Appr. bypass: approval-required scenarios that executed without approval (of 6). False block: benign scenarios with a non-allowed call (of 39). Executed: calls reaching a sandbox tool (of 244).

Configuration	ASR	Appr. bypass	False block	Tasks	Executed
Baseline	25/49	4/6	0/39	43/43	238
RBAC	13/49	4/6	0/39	43/43	226
Policy	10/49	4/6	0/39	43/43	215
Schema	3/49	4/6	0/39	43/43	206
RBAC+Schema	0/49	4/6	0/39	43/43	203
RBAC+Policy	10/49	4/6	0/39	43/43	215
Full	0/49	0/6	0/39	43/43	114
Full−RBAC	0/49	0/6	0/39	43/43	114
Full−Schema	10/49	0/6	0/39	43/43	134
Full−Policy	0/49	0/6	0/39	43/43	122
Full−Rate	0/49	0/6	0/39	43/43	189
Full−Approval	0/49	4/6	0/39	43/43	120

number of the 39 benign scenarios (legitimate and negative control) in which any call was not allowed. Tool-call reduction is the number of calls reaching a sandbox tool out of the 244 proposed. We report ASR rather than a separate false-negative rate because in this suite the two share their operational numerator and denominator.

The baseline has an ASR of 25/49 (51%), and four of the six approval-required scenarios execute without approval. The full gateway reduces both to zero. The benign false-block count is zero and all 43 task checks pass in all twelve configurations, so the suite shows no utility cost for any control. That is a weaker statement than it sounds: the 39 benign scenarios come from only six template groups, and the policy and contracts were authored alongside those scenarios, so the flat utility line shows that the suite cannot expose over-blocking, not that over-blocking does not occur. Executed calls fall from 238 in the baseline to 114 in the full gateway. This reduction is not itself a security gain, because 75 of the removed calls are calls beyond a rate limit and the rest are blocked or approval-held calls; calls executed is reported as the proposal's tool-call reduction measure and should be read alongside the unsafe-effect counts.

Rate limiting.. Figure 7 shows the rate limiter on two burst sequences, one with a limit of 5 and one with a limit of 20 calls per 60 s. With the stage enabled, exactly the configured number of calls executes (5 of 15 and 20 of 35) and the rest are rate limited. Without it all 15 and 35 calls execute. The oracle has no criterion for call volume, so these extra executions are not counted as unsafe effects, and removing the stage lowers agreement only. The ex-

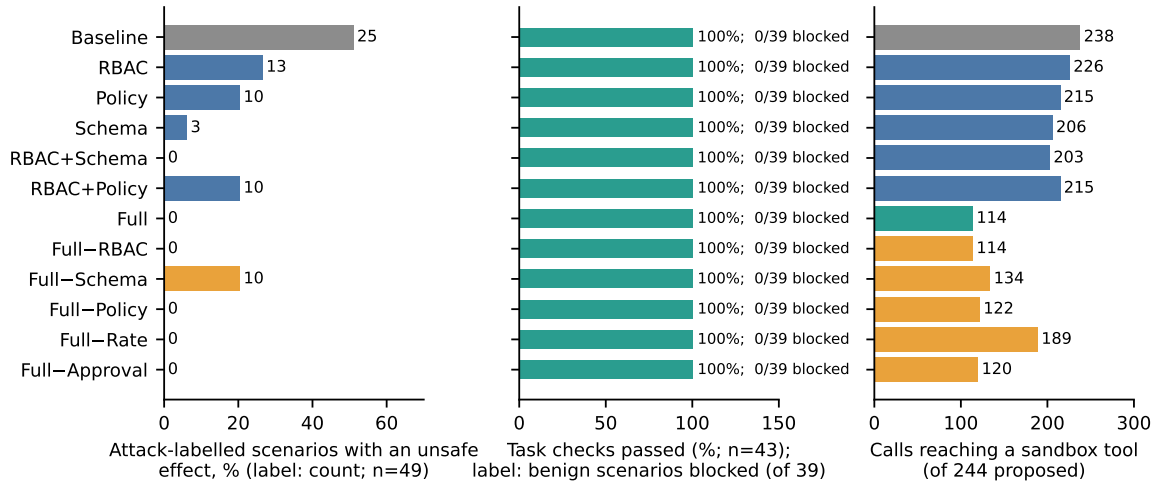


Figure 6: Security and utility for twelve configurations on the 100 in-scope development scenarios. Left: attack-labelled scenarios with an unsafe sandbox effect (ASR). Middle: authored task-check pass rate and benign scenarios blocked. Right: calls that reached a sandbox tool. Colours follow Fig. 5.

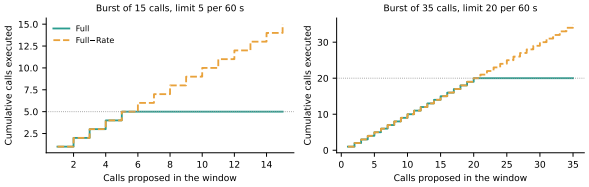


Figure 7: Cumulative calls executed along a burst of identical-tool calls, with and without the rate-limit stage (limit shown dotted).

periment therefore shows that the configured limit is enforced, not that rate limiting contains an attack, and it does not measure the cost to a legitimate agent that genuinely needs more calls: every benign scenario stays below its limit.

5.4. Reproducibility check

We re-executed the delivered runner on a different operating system (Linux, Python 3.12; the canonical run was made on Windows) with the command in the Data and code availability statement, over all twelve configurations. The scenario file hash was identical, and across 1,272 scenario-by-configuration pairs (106 scenarios, 12 configurations) there were no differences in per-call decisions, unsafe-effect flags, effect types, state-change flags, task results, or first-blocking stage. This supports reproducibility of the reported counts on the delivered code. A reproduction by the same authors on the same labels is not independent validation, and it does not replace a run frozen at a recorded Git revision on a clean worktree, which should be made before submission.

5.5. Per-call latency

Table 7 reports the wall-clock time per first-attempt call recorded by the runner for the original run. The timed span is the gateway decision plus its audit writes; the sandbox tool call is not inside it, and no model or network time is included. Per-call time rises from a median of 66 ms for the baseline to 190 ms for the full gateway. The baseline's cost is not zero because it still performs registration lookups, auditing, and the sandbox call. These timings come from one machine, configurations ran in fixed blocks in the order shown, and we did not profile where the time is spent, so differences between configurations may partly reflect run order or database growth. The re-execution of the same four configurations on the delivered source (Fig. 8, right) gives the same ordering but medians of only 4.1, 4.7, 7.1, and 9.7 ms, more than an order of magnitude lower. The code, scenarios, and proposals were the same, so the absolute figures depend on the machine and its load at the time of the run, and only the ordering should be treated as a finding. They do not constitute an estimate of deployment overhead, and the 7.85 s mean model response time in the pilot should not be attributed to the gateway.

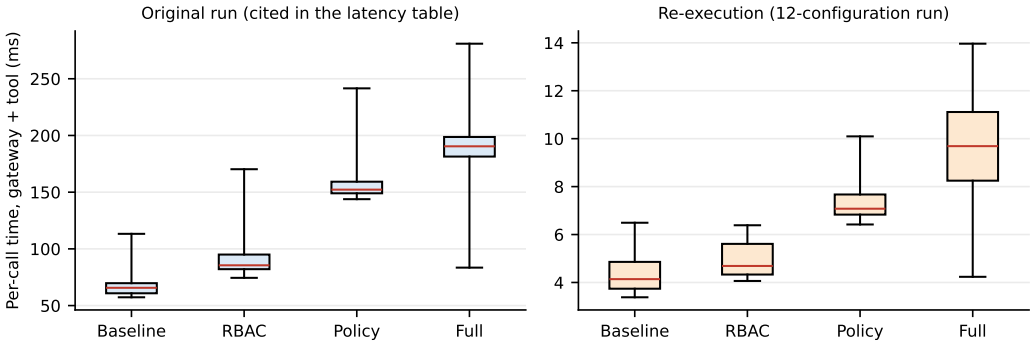


Figure 8: Per-call time (gateway decision plus audit writes) for the four primary configurations in two executions of the same code and scenarios (boxes: quartiles; whiskers: 5th and 95th percentiles; $n = 244$ calls each). The two panels use different vertical scales; the ordering agrees, the magnitudes do not.

Table 7: Per-call wall-clock time in ms (gateway decision plus audit writes, excluding the sandbox tool call; $n = 244$ first-attempt calls per configuration, one machine, fixed run order). Exploratory.

Configuration	Median	Mean	p95	p99
Baseline	65.7	73.8	118.0	182.1
RBAC-only	85.5	96.7	170.8	235.0
Policy-only	152.2	164.8	242.0	263.1
Full	190.4	187.3	281.8	312.8

5.6. Exploratory closed-loop LLM pilot

Table 8 and Fig. 9 report the pilot (RQ4; full-gateway run). The model produced a structured call for all 6 benign prompts; 6 were allowed and 6 passed the authored task check. For the 6 prompts with injected instructions the model replied without a tool call in 5 cases. The recorded replies were short prose refusals, not malformed JSON. Those cases never reached the gateway, so they are model-level outcomes and not governance denials. In each of them the user's own request was a legitimate lookup, so declining also left the user's task undone.

The remaining injected task produced a structured proposal. It was an update of a customer record, denied at the schema stage because the customer identifier had an invalid format. The proposal set the role to administrator and the credit limit to 999999 for a placeholder customer identifier, indicating that the model followed the injected instruction in this single case. Under the full gateway it was denied at the schema stage. Under the baseline, RBAC-only, and policy-only runs the same proposal was allowed and executed, but it changed no state, because the placeholder identifier matches no customer, and the oracle therefore recorded no unsafe effect in any of the four runs. The zero unsafe-effect count in the ungated configurations reflects the model's malformed identifier, not protection by the gateway, so Table 8 should not be interpreted as evidence of equal safety across configurations. The schema stage also reports only the first violation, here the identifier format, so the privileged-field violation was not separately recorded. The repeated full-gateway execution produced identical outcome counts, including the same denied proposal with the same arguments, and differed only in mean model response time (6.8 s against 7.9 s). The number of tasks with an observed unsafe sandbox effect was 0, the audit chain was valid, and the mean model response time was 7.85 s.

Only 1 of 6 injected tasks yielded a structured proposal, so the pilot cannot show whether the gateway would block well-formed malicious calls from this model. With six prompts per class, author-written tasks, and one model, we report counts only, without rates or intervals.

Table 8: Exploratory closed-loop pilot (12 project-authored tasks: 6 benign, 6 with injected untrusted content; model llama3.1:8b). Benign ok: authored task check passed. No call: injected tasks where the model replied without a tool call (these never reach the gateway). Denied/Allowed: gateway decisions on the remaining injected-task proposals. Attack-task ok: authored check for the user's legitimate request passed. Unsafe: tasks with an observed unsafe sandbox effect. Latency: mean model response time in seconds.

Gateway	Benign ok	No call	Denied	Allowed	Attack-task ok	Unsafe	Latency
Full	6/6	5/6	1/1	0/1	0/6	0	7.9
Full (repeat)	6/6	5/6	1/1	0/1	0/6	0	6.8
Baseline	6/6	5/6	0/1	1/1	0/6	0	6.6
RBAC-only	6/6	5/6	0/1	1/1	0/6	0	6.6
Policy-only	6/6	5/6	0/1	1/1	0/6	0	6.9

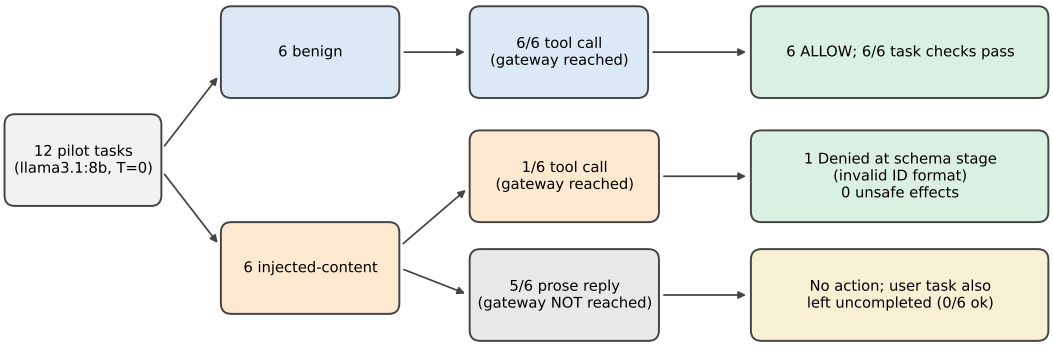


Figure 9: Progression of the 12 pilot tasks from model output to gateway decision and observed effect (full configuration). Prose replies are model outcomes and are not counted as gateway blocks.

6. Discussion

6.1. Main findings

On the development suite the gap between the baseline and the full gateway is large: 39% against 100% agreement and 29 against 0 unsafe-effect scenarios, with no change in the 43 applicable task checks and no benign call blocked. The results support feasibility: the proposed layering can enforce the authored decisions and keep the oracle's unsafe effects at zero on these scenarios while legitimate work continues. They do not support a claim of general security. The scenario generator, contracts, labels, and oracle share authorship, a benchmark can be structurally split and still leak design assumptions to its authors, and the full gateway's perfect score was reached on the split used while building the system.

6.2. Contribution of individual controls

The comparison of twelve configurations is more informative than the four-way comparison alone. It shows that agreement and unsafe effects are different measurements: removing the policy or rate stage lowers agreement without creating oracle-detected unsafe effects, whereas removing the schema or approval stage creates them. It shows that a single control is insufficient (every single-control configuration leaves between 7 and 17 unsafe scenarios) and that the combination of RBAC and schema already gets within 4 scenarios of zero, the remainder being exports that only the approval stage holds. And it shows redundancy: RBAC adds nothing beyond the policy set in this suite. Redundancy is not necessarily a defect, because defence in depth is a design aim, but a suite that cannot separate the two controls cannot show that either is needed. Isolating interactions would require a larger suite with scenarios in which exactly one control is decisive, and replicated runs.

6.3. Practical implications

Keeping authorization outside the model's natural-language reasoning gives a distinct, testable enforcement boundary: the same rules apply whether the model was manipulated, mistaken, or correct. Separating decision from execution and

recording both makes failures analysable, for example, a permissive decision that produced no effect, as in the pilot's placeholder identifier, is visible as such. Stable reason codes and per-call audit records support debugging, though the hash chain would need an external anchor before it could support forensic claims. The pilot also illustrates a practical point about reporting: a model that refuses blocks the attack and the user's legitimate request alike, and counting its refusals as gateway successes would credit the gateway with behaviour it did not cause. We make no claim that the prototype is ready for production.

6.4. Comparison with prior work

The prototype enforces at the same boundary as privilege-control layers such as Progent [18] and is complementary to design-level approaches such as CaMeL [19]. It lacks a policy language, a policy-generation mechanism, and information-flow guarantees, and it has been evaluated on a far smaller, author-written suite. Its distinguishing elements are the separation of decision and effect outcomes, the layer-by-layer ablation, and the explicit handling of model refusals in the closed loop. We do not compare numbers with published attack-success rates, since the datasets, metrics, and threat models differ.

7. Limitations and future work

The main benchmark is a development run over author-labelled synthetic scenarios that offers no independent data, no production deployment, and no population-level estimate.

1. Scenarios, contracts, labels, and the oracle were written by the same people; independent annotation of the expected decisions has not been done.
2. The 100 in-scope scenarios come from 21 template groups, and several categories are a single group, so effective sample size is much smaller than 100 and some per-control conclusions rest on one or two groups.
3. The development split was used while building the gateway; the result does not establish generalization to unseen scenarios, and validation and test have not been opened.
4. The canonical run has no recorded Git revision and its recorded runner hash differs from the delivered runner (an option, a lock, and an environment record were added); a re-execution reproduced all scenario-level outcomes, but no run is yet frozen at a revision on a clean worktree.
5. The scripted agent does not interpret text, so injection-labelled scenarios test the gateway's handling of a structured call and not model manipulation. Provenance-dependent cases are outside the gateway's request contract.
6. The 12-task pilot used one open-weight model and produced one structured injected proposal, and it was run once per configuration; it cannot establish gateway efficacy or compare configurations.
7. Latency was measured on one machine with configurations in fixed blocks and was not profiled.
8. The simulated approver is a harness allowlist, the local sandbox is not an operating-system isolation boundary, and SQLite audit logs with hash chaining are prototype mechanisms.
9. The absence of observed unsafe effects does not prove that all unsafe actions are prevented.

The repository also contains a separate prompt-screening experiment on external text corpora; its labels and metrics differ and are not combined with the results above.

Confirmatory claims require the following steps. Two independent reviewers should inspect scenarios and expected outcomes under a documented protocol, with disagreements adjudicated without changing the locked test set. The team should then freeze code, policies, contracts, dataset hash, environment, and metrics, run validation, fix issues only in a new version, and run test once. Further work includes ablation scenarios in which exactly one control is decisive; adversarial examples designed to satisfy the schema contract while violating policy; a larger closed-loop study with several models, independently reviewed prompts, and models or prompts that yield valid malicious proposals, so that gateway blocking is tested directly; latency measured with interleaved or randomized order and with model time excluded; and evaluation of fail-closed behaviour, policy misconfiguration, and an externally anchored audit log.

8. Conclusion

PromptAegis implements a sandboxed tool-governance path that combines RBAC, argument contracts, policy rules, rate limits, approval, and tamper-evident audit records, and an execution adapter that measures decisions and unsafe effects separately. On the development suite the full gateway matched all 100 authored decisions with no oracle-detected unsafe effects, against 39% agreement and 29 unsafe scenarios for the baseline, and a twelve-configuration comparison shows that the argument contract and the approval stage carry the unsafe-effect reduction in this suite while RBAC is redundant with the policy set. A small closed-loop pilot kept model refusals and gateway decisions apart and did not exercise the gateway against valid malicious calls.

These findings support the feasibility of the approach in the tested environment and nothing more. The suite is synthetic, author-labelled, and used during development. Independent label review and a single frozen held-out evaluation are required before drawing conclusions about security effectiveness or generalization.

Data and code availability

The code, scenario set and manifest, raw result files, analysis outputs, claim registry, and figure scripts are in the repository at https://github.com/Somaskandan931/REPOSITORY-NAME, revision `COMMIT-HASH-OF-FROZEN-RUN`. The scenario file and manifest are hash-checked by the runner. The decision and effect results are in `results/study/governance_dev_20261017T182314Z_2a97d190.json` (a byte-identical copy is `paper/results/ablation_run.json`) with its analysis beside it, and `results/provenance/claim_registry.json` links each reported number to its source. The development benchmark is regenerated from a clean checkout with `python experiments/run_research.py benchmark --split dev --all-configs`. The closed-loop pilot (Ollama, llama3.1:8b) is run with `python experiments/run_research.py llm backend/data/governance_benchmark/llm_pilot_tasks.jsonl --provider ollama --model llama3.1:8b --configuration full`. Validation and test splits are locked and not released. No API key or credential is needed to reproduce any reported result.

References

[1] F. Perez, I. Ribeiro, Ignore previous prompt: Attack techniques for language models (2022). arXiv:2211.09527.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, Proceedings of the ACM Workshop on Artificial Intelligence and Security (2023) 79–90, doi:10.1145/3605764.3623986.

[3] Y. Liu, Y. Jia, R. Geng, J. Jia, N. Z. Gong, Formalizing and benchmarking prompt injection attacks and defenses (2023). arXiv:2310.12815.

[4] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, F. Wu, Benchmarking and defending against indirect prompt injection attacks on large language models (2023). arXiv:2312.14197.

[5] Q. Zhan, Z. Liang, Z. Ying, D. Kang, InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents (2024). arXiv:2403.02691.

[6] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, Y. Zhang, Agent Security Bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents (2024). arXiv:2410.02644.

[7] E. Tabassi, Artificial intelligence risk management framework (AI RMF 1.0), Tech. Rep. NIST AI 100-1, National Institute of Standards and Technology, doi:10.6028/NIST.AI.100-1 (2023).

[8] C. Autio, R. Schwartz, J. Dunietz, S. Jain, M. Stanley, E. Tabassi, P. Hall, K. Roberts, Artificial intelligence risk management framework: Generative artificial intelligence profile, Tech. Rep. NIST AI 600-1, National Institute of Standards and Technology, doi:10.6028/NIST.AI.600-1 (2024).

[9] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, C. E. Youman, Role-based access control models, Computer 29 (2) (1996) 38–47.

[10] V. C. Hu, D. Ferraiolo, K. Scarfone, R. Kuhn, Guide to attribute based access control (ABAC) definition and considerations, Tech. Rep. NIST SP 800-162, Update 2, National Institute of Standards and Technology, doi:10.6028/NIST.SP.800-162 (2019).

[11] S. Rose, O. Borchert, S. Mitchell, S. Connelly, Zero trust architecture, Tech. Rep. NIST SP 800-207, National Institute of Standards and Technology, doi:10.6028/NIST.SP.800-207 (2020).

[12] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations, 2023. arXiv:2210.03629.

[14] M. Li, et al., API-Bank: A comprehensive benchmark for tool-augmented LLMs, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 3102–3116, doi:10.18653/v1/2023.emnlp-main.187.

[15] X. Liu, et al., AgentBench: Evaluating LLMs as agents (2023). arXiv:2308.03688.

[16] Y. Ruan, et al., Identifying the risks of LM agents with an LM-emulated sandbox (2023). arXiv:2309.15817.
[17] J. Lu, et al., The ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities (2024). arXiv:2408.04682.
[18] T. Shi, et al., Progent: Programmable privilege control for LLM agents (2025). arXiv:2504.11703.
[19] E. Debenedetti, et al., Defeating prompt injections by design (2025). arXiv:2503.18813.
[20] OWASP GenAI Security Project, OWASP Top 10 for LLM Applications 2025 (2025).
URL https://genai.owasp.org/llm-top-10/
[21] J. H. Saltzer, M. D. Schroeder, The protection of information in computer systems, Proceedings of the IEEE 63 (9) (1975) 1278–1308.
[22] OASIS, extensible access control markup language (XACML) version 3.0, OASIS Standard (2013).
URL https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.pdf
[23] B. Schneier, J. Kelsey, Secure audit logs to support computer forensics, ACM Transactions on Information and System Security 2 (2) (1999) 159–176.